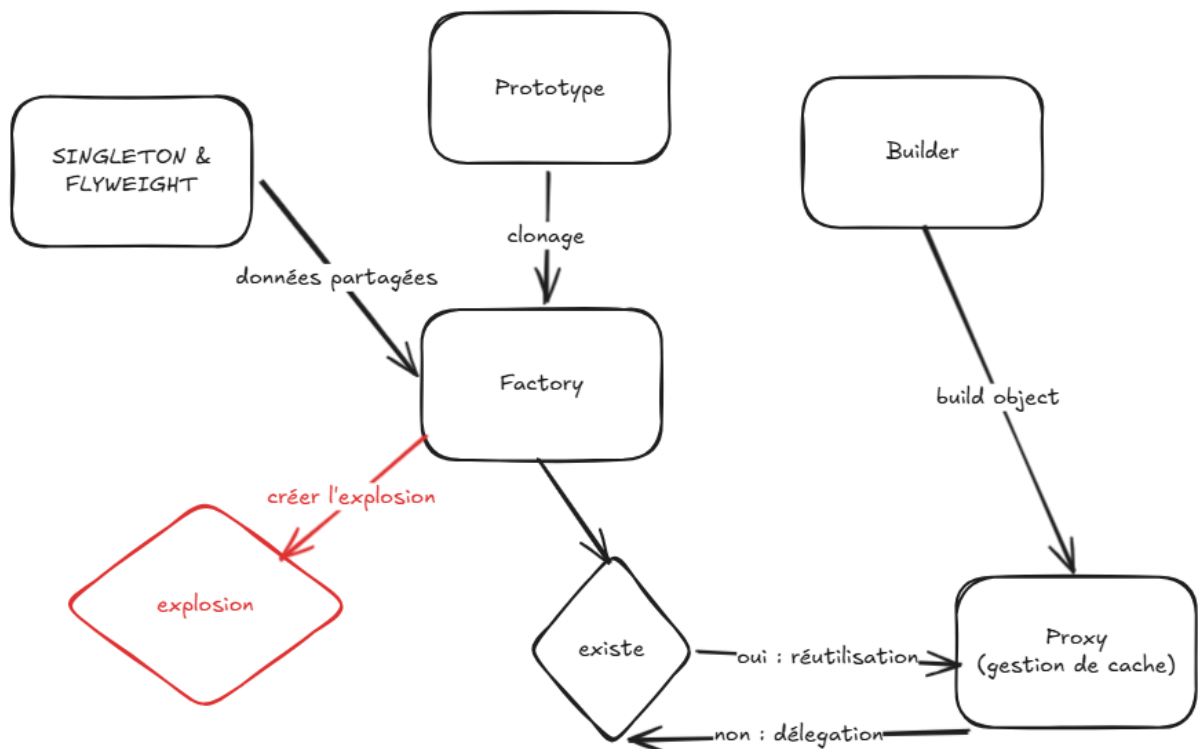


## TP Design patterns

### Système d'Explosions de Particules

Ce projet présente un moteur d'explosions modulaire des particules. Le Builder prépare l'explosion avec les configs choisies. Le Proxy vérifie s'il l'a déjà pour économiser la mémoire, sinon il appelle la fabrique. La fabrique crée l'explosion en partageant les données (Flyweight) et en clonant une particule (Prototype).

Schéma de système :



Le code :

```
#include <iostream>
#include <string>
#include <vector>

class IParticleFlyweight
{
```

```

public:
    virtual void afficher() = 0;
    virtual ~IParticleFlyweight() = default;
};

//concret
class ParticleFlyweight : public IParticleFlyweight
{
private:
    std::string texture;
    std::string shader;
    std::string compPhysiques;

public:
    ParticleFlyweight(const std::string& type)
    {
        this->texture = "texture" + type + ".png";
        this->shader = "shade" + type + ".png";
        this->compPhysiques = "compPhysique" + type;
    }

    void afficher() override
    {
        std::cout << "donnees : texture: " << texture << " et Shader: " <<
shader << ")\n";
    }
};

class FlyweightFactory
{
private:
    std::vector<std::string> cles;
    std::vector<IParticleFlyweight*> flyweights;
    static FlyweightFactory* instance;
    FlyweightFactory()
    {
        std::cout << "initialisation.\n";
    }

public:
    static FlyweightFactory* recupereInstance()
    {
        if (instance == nullptr)
        {
            instance = new FlyweightFactory();
        }
        return instance;
    }
};

```

```

    }

    IParticleFlyweight* creationFlyweight(const std::string& flyParCle)
    {
        //si le flyweight existe déjà
        for (size_t i = 0; i < cles.size(); i++)
        {
            if (cles[i] == flyParCle)
            {
                std::cout << "reutilisation de flyweight : " << flyParCle <<
"\n";
                return flyweights[i];
            }
        }

        std::cout << "Creation de flyweight\n";
        IParticleFlyweight* nvFlyweight = new ParticleFlyweight(flyParCle);

        cles.push_back(flyParCle);
        flyweights.push_back(nvFlyweight);

        return nvFlyweight;
    }

    ~FlyweightFactory()
    {
        for (size_t i = 0; i < flyweights.size(); i++)
        {
            delete flyweights[i];
        }
    }
};

FlyweightFactory* FlyweightFactory::instance = nullptr;

class ExplosionConfig
{
private:
    float x;
    float y;
    std::string couleur;
    int nombre;
    float spread;
    float dureeDeVie;

public:
    ExplosionConfig(float x, float y, std::string couleur, int nombre, float
spread, float dureeDeVie)
    {

```

```

        this->x = x;
        this->y = y;
        this->couleur = couleur;
        this->nombre = nombre;
        this->spread = spread;
        this->dureeDeVie = dureeDeVie;
    }

    float recupereX()
    {
        return x;
    }
    float recupereY()
    {
        return y;
    }
    std::string recupereCouleur()
    {
        return couleur;
    }
    int recupereNombre()
    { return nombre;
    }
    float recupereSpread()
    {
        return spread;
    }
    float recupereDureeDeVie()
    {
        return dureeDeVie;
    }

    void afficherConfig()
    {
        std::cout << "explosion dans localisation : "<< x << ", " << y << " avec
"<< nombre << "particules de couleur : "<< couleur << ".\n";
    }
};

class IParticlePrototype
{
public:
    virtual IParticlePrototype* clone() = 0;
    virtual void afficher() = 0;
    virtual ~IParticlePrototype() = default;
};

//prototype concret
class ParticlePrototype : public IParticlePrototype

```

```

{
private:
    float taille;
    std::string couleur;
    float vitesse;

public:
    ParticlePrototype(float taille, std::string couleur, float vitesse)
    {
        this->taille= taille;
        this->couleur = couleur;
        this->vitesse = vitesse;
    }

    ParticlePrototype(const ParticlePrototype& autre)
    {
        this->taille = autre.taille;
        this->couleur = autre.couleur;
        this->vitesse = autre.vitesse;
    }

    IParticlePrototype* clone() override
    {
        return new ParticlePrototype(*this);
    }

    void afficher() override
    {
        std::cout << "Particule avec taille: " <<taille<<" et couleur: " <<
couleur<< " et vitesse: "<<vitesse<<"\n";
    }
};

class ExplosionBuilder
{
private:
    float x;
    float y;
    std::string couleur;
    int nombre;
    float spread;
    float dureeDeVie;

public:
    ExplosionBuilder()
    {
        //initialisation avec des valeurs par défaut
        this->x = 0.0f;
        this->y = 0.0f;
    }
};

```

```

        this->couleur = "Blanc";
        this->nombre = 0;
        this->spread = 0.0f;
        this->dureeDeVie = 0.0f;
    }

    ExplosionBuilder* withPosition(float x, float y)
    {
        this->x = x;
        this->y = y;
        return this;
    }

    ExplosionBuilder* withColor(std::string couleur)
    {
        this->couleur = couleur;
        return this;
    }

    ExplosionBuilder* withCount(int nombre)
    {
        this->nombre = nombre;
        return this;
    }

    ExplosionBuilder* withSpread(float spread)
    {
        this->spread = spread;
        return this;
    }

    ExplosionBuilder* withLifetime(float dureeDeVie)
    {
        this->dureeDeVie = dureeDeVie;
        return this;
    }

    //création et build de l'objet
    ExplosionConfig* build()
    {
        return new ExplosionConfig(this->x, this->y, this->couleur, this->nombre, this->spread, this->dureeDeVie);
    }
};

//particule qui va englober tout
class Particle
{
private:

```

```

    IParticlePrototype* prototype;
    IParticleFlyweight* flyweight; //partagée
    float x, y;
    float vitX, vitY;

public:
    Particle(IParticlePrototype* proto, IParticleFlyweight* fly, float x,
float y, float vitX, float vitY)
    {
        this->prototype = proto;
        this->flyweight = fly;
        this->x = x;
        this->y = y;
        this->vitX = vitX;
        this->vitY = vitY;
    }

    ~Particle()
    {
        delete prototype;
    }

    void afficher()
    {
        std::cout << "Particule a la position : "<<x<<","<<y<< " avec vitX/vitY
:"<<vitX<< "/"<<vitY<<"\n";
        prototype->afficher();
        flyweight->afficher();
    }
};

class Explosion
{
private:
    std::vector<Particle*> particules;

public:
    void ajouterParticule(Particle* p)
    {
        particules.push_back(p);
    }

    void render()
    {
        std::cout << "Explosion avec : "<<particules.size()<< " particules\n";
        for (size_t i = 0; i < particules.size(); i++)
        {
            particules[i]->afficher();
        }
    }
};

```

```

    }

    ~Explosion()
    {
        for (size_t i = 0; i < particules.size(); i++)
        {
            delete particules[i];
        }
    }
};

class IExplosionFactory
{
public:
    virtual Explosion* createExplosion(ExplosionConfig* config) = 0;
    virtual ~IExplosionFactory() = default;
};

//factory concret pour créer une explosion complete
class ExplosionFactory : public IExplosionFactory
{
public:
    Explosion* createExplosion(ExplosionConfig* config) override
    {
        Explosion* explosion = new Explosion();

        FlyweightFactory* fwFactory = FlyweightFactory::recupereInstance();
        IParticleFlyweight* flyweight = fwFactory->creationFlyweight(config->recupereCouleur());
        ParticlePrototype pp(1.0f, config->recupereCouleur(), 10.0f);

        int nombre = config->recupereNombre();
        float spread = config->recupereSpread();

        for (int i = 0; i < nombre; i++)
        {
            IParticlePrototype* ppClone = pp.clone();

            //vitesse aleatoire
            float vitX = spread * (0.5f - ((i % 3) * 0.20f));
            float vitY = spread * (0.5f - ((i % 2) * 0.30f));

            Particle* nvParticule = new Particle(ppClone, flyweight, config->recupereX(), config->recupereY(), vitX, vitY);

            explosion->ajouterParticule(nvParticule);
        }

        return explosion;
    }
};

```



```

    }
};

//proxy
class ExplosionCacheProxy : public IExplosionFactory
{
private:
    ExplosionFactory* ef;
    std::vector<ExplosionConfig*> cacheDesConfigs;
    std::vector<Explosion*> cacheDesExplosions;

public:
    ExplosionCacheProxy()
    {
        ef = new ExplosionFactory();
    }

    ~ExplosionCacheProxy()
    {
        delete ef;
        //pour nettoyer les explosions
        for (size_t i = 0; i < cacheDesExplosions.size(); i++)
        {
            delete cacheDesExplosions[i];
        }
    }

    Explosion* createExplosion(ExplosionConfig* config) override
    {
        //les configs en memoire
        for (size_t i = 0; i < cacheDesConfigs.size(); i++)
        {
            if (cacheDesConfigs[i]->recupereCouleur() == config-
>recupereCouleur() &&
                cacheDesConfigs[i]->recupereNombre() == config-
>recupereNombre())
            {
                std::cout << "explosion deja en memoire\n";
                return cacheDesExplosions[i];
            }
        }

        //sinon
        std::cout << "delegation au factory\n";
        Explosion* nvExplosion = ef->createExplosion(config);

        //enregistre le cache
        cacheDesConfigs.push_back(config);
        cacheDesExplosions.push_back(nvExplosion);
    }
};

```

```

        return nvExplosion;
    }
};

int main()
{
    FlyweightFactory* fwFactory = FlyweightFactory::recupereInstance();

    IParticleFlyweight* flyweight = fwFactory->creationFlyweight("flyweight1");
    flyweight->afficher();

    //exemple si le flyweight existe déjà
    IParticleFlyweight* fw2 = fwFactory->creationFlyweight("flyweight1");
    fw2->afficher();

    //utilisation de la pattern builder pour créer un objet
    std::cout << "creation de l'objet avec Builder : \n";
    ExplosionBuilder* objBuilder = new ExplosionBuilder();
    ExplosionConfig* config = objBuilder->withPosition(10.0f, 20.0f)
        ->withColor("rouge")
        ->withCount(10)
        ->withSpread(20.0f)
        ->withLifetime(5.0f)
        ->build();

    config->afficherConfig();

    std::cout << "creation du prototype : \n";
    IParticlePrototype* pp = new ParticlePrototype(2.5f, config->recupereCouleur(), 15.0f);
    pp->afficher();

    std::cout << "test clone prototype pour l'explosion : \n";
    IParticlePrototype* ppClone1 = pp->clone();
    IParticlePrototype* ppClone2 = pp->clone();
    ppClone1->afficher();
    ppClone2->afficher();

    std::cout << "test de la factory : \n";
    IExplosionFactory* fExplosion = new ExplosionFactory();
    Explosion* testExplosion = fExplosion->createExplosion(config);
    testExplosion->render();

    std::cout << "\ntest proxy : \n";
    IExplosionFactory* fProxy = new ExplosionCacheProxy();

    std::cout << "demande d'explosion : \n";

```

```
Explosion* explosion1 = fProxy->createExplosion(config);
explosion1->render();

std::cout << "demande d explosion avec les memes configs :\n";
Explosion* explosion2 = fProxy->createExplosion(config);
explosion2->render();

delete fwFactory;
delete objBuilder;
delete config;
delete pp;
delete ppClone1;
delete ppClone2;
delete fExplosion;
delete testExplosion;
delete FlyweightFactory::recupereInstance();
delete fProxy;

return 0;
}
```